

Deliverable 1 — High-Level Design Document

Project: CivicLink — Citizen Requests & Complaints Platform **Workshop:** Lean (post agile) Planning and Budgeting Strategies **Course:** Enterprise Architectures — ECI **Authors:** Andersson D. Sánchez M., Cristian S. Pedraza R., Raquel I. Selma A.

1. Executive Summary

CivicLink is a web-based platform that lets citizens of a mid-sized Colombian municipality submit, track, and resolve service requests (PQRS) — public lighting outages, missed waste collection, damaged infrastructure, noise complaints, and similar — and lets municipal staff assign, work, and report on those requests. The architecture is **microservices-based, event-driven, cloud-native, and integrable with the existing public IT stack** (government SSO, GIS, open-data portals).

This document describes the system through the **Digital City Hall metaphor** (Section 2), then formalises it into a high-level architecture (Sections 3–6) with views for stakeholders.

2. The System Metaphor — Digital City Hall

XP's **system metaphor** is a shared story that helps everyone — developers, mayors, citizens — talk about the system using the same vocabulary. CivicLink is a **Digital City Hall**: every interaction has a counterpart in a real municipal building.

A citizen walks into the **Lobby** (Citizen Portal), is greeted at the **Reception** (API Gateway), shows their **ID** at the ID Office (Identity Service), files their request at the **Filing Office** (Requests Service), and gets a receipt in the **Mail Room** (Notification Service). City staff enter through the **Front Desk** (Admin Dashboard), consult the **Directory** (Department Service) to know who handles what, and the **Auditor's Office** (Reports Service) tells the mayor how the building is performing.

Why this metaphor works:

- **Concreteness.** Every component has a physical counterpart, which makes design conversations grounded.
- **Modularity by analogy.** Real city halls are organised by department, each with its own files and staff — that maps naturally to microservices.
- **Resilience by analogy.** If the Mail Room is overloaded, the Filing Office still works — exactly how an event-driven architecture isolates failures.

- **Onboarding.** New developers, designers, and even non-technical product owners can reason about the system within minutes.

2.1 Metaphor → Component Mapping

Real City Hall	CivicLink Component	Technology	Single Responsibility
The Lobby	Citizen Portal	React + Vite + Tailwind, PWA	Public-facing UI for citizens
The Front Desk	Admin Dashboard	React + shadcn/ui + Recharts	Internal UI for staff and admins
The Reception	API Gateway	Kong / AWS API Gateway	Routing, JWT validation, rate limiting, CORS
The ID Office	Identity Service	Auth0 + Spring Boot facade	Auth, profiles, roles
The Filing Office	Requests Service	Spring Boot + PostgreSQL	CRUD + lifecycle of requests
The Directory	Department Service	Spring Boot + PostgreSQL	Departments, staff, assignment rules
The Mail Room	Notification Service	Spring Boot + SendGrid + Twilio + FCM	Email, SMS, push, in-app
The Auditor's Office	Reports & Analytics Service	Spring Boot + Big-Query/Redshift	KPIs, dashboards, exports
The Photo Wall	Media & Geo Service	Spring Boot + S3 + Maps API	Photos, attachments, geocoding
The Courier	Event Bus	Apache Kafka (MSK)	Async messages between services
The Vault	Identity DB	PostgreSQL (managed)	Citizen credentials and roles
The Filing Cabinet	Requests DB	PostgreSQL + PostGIS	Requests, assignments, status history
The Archive	Analytics Warehouse	BigQuery / Redshift	Historical reporting data
The Postal Service	Email/SMS/Push Gateways	SendGrid, Twilio, FCM	External delivery channels
The City Map	Maps Provider	Google Maps / OpenStreetMap	Geocoding, tiles, heatmaps

2.2 Metaphor Diagram

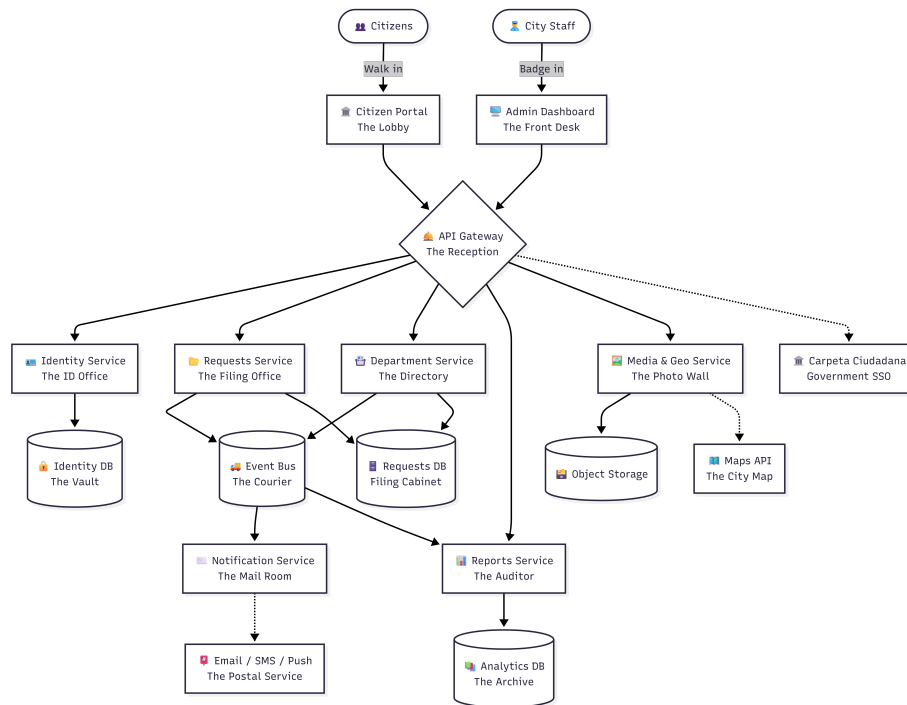


Figure 1: Digital City Hall Metaphor Diagram

3. Architectural Drivers

The architecture is shaped by these quality attributes (in priority order):

#	Driver	Concrete Target	How the Architecture Addresses It
1	Availability	99.5% on Citizen Portal, 99.9% on submission flow	Multi-AZ PostgreSQL, stateless services, event-driven decoupling

#	Driver	Concrete Target	How the Architecture Addresses It
2	Modularity	Independent deployability of every service	Microservices, contract-first APIs, separate repos or monorepo with isolated CI
3	Scalability	5,000 concurrent citizens, 500 requests/min peak	Horizontal pod autoscaling, Redis cache, Kafka buffering
4	Security	OWASP Top 10, GDPR/Habeas Data, audit trail	OIDC + RS256 JWT, encryption at rest/in transit, immutable audit log in MongoDB
5	Integrability	Open data API, government SSO, GIS layer	Public REST + OpenAPI, OIDC federation, webhook-based integrations
6	Observability	Mean time to detect < 5 min	Prometheus metrics, ELK logs, Sentry errors, distributed tracing (OpenTelemetry)
7	Accessibility	WCAG 2.1 AA on Citizen Portal	Semantic HTML, ARIA, keyboard nav, screen-reader testing
8	Cost-efficiency	Cloud bill < USD 1,500/month at MVP scale	PAY_PER_REQUEST DynamoDB-style services where possible, autoscale to zero on dev

4. Logical Architecture

4.1 Service Catalog

Service	Responsibility	Storage	Sync API	Async (events emitted / consumed)
identity-service	Authn, authz, profiles	PostgreSQL (users, roles)	/auth/*, /me, /users/*	emits user.created, user.updated
requests-service	Requests lifecycle	PostgreSQL + PostGIS	/requests/*, /stream	emits request.created, request.statusChanged, request.assigned
departments-service	Departments, staff, rules	PostgreSQL	/departments/*, /assignments/*	consumes request.created to auto-assign
media-service	Photos, attachments, geocoding	S3 + MongoDB metadata	/media/upload,— /media/{id}	—
notifications-service	Email, SMS, push, in-app	MongoDB (delivery log)	/notifications/*	consumes request.*, user.created
reports-service	KPIs, dashboards, exports	Warehouse + Redis	/reports/*, /kpis/*	consumes all request.* events for ETL

4.2 Logical Diagram

5. Deployment Architecture

CivicLink runs on a **Kubernetes** cluster (EKS or AKS) inside a single cloud VPC, with managed data services and a CDN at the edge.

5.1 Environments

Environment	Purpose	Data	Cost band
dev	Daily development, ephemeral	Synthetic	\$300/month

Environment	Purpose	Data	Cost band
staging	UAT, performance, security tests	Anonymised production-like	\$700/month
prod (out of project scope)	Live citizens	Real	budget separately as ops

6. Cross-Cutting Concerns

6.1 Security

- **Authentication.** OIDC + RS256 JWT issued by Auth0; tokens validated by every service via JWKS.
- **Authorization.** Role-based: `citizen`, `staff`, `dept-head`, `admin`, `auditor`. Enforced at the gateway and re-checked at each service.
- **Transport.** TLS 1.2+ everywhere. Certificates managed by ACM/Let's Encrypt.
- **At rest.** AES-256 on RDS, S3, EBS volumes. Photos uploaded via presigned URLs.
- **Audit.** Every state change appended to an immutable MongoDB collection.
- **Data privacy.** GDPR / Colombian *Habeas Data* (Ley 1581) — citizens can export and delete their data; PII minimised in logs.
- **OWASP.** Standard mitigations: parameterised queries, input validation, output encoding, CSP headers, dependency scanning in CI.

6.2 Observability

- **Metrics.** Prometheus scrapes every service; Grafana dashboards per microservice.
- **Logs.** Structured JSON, shipped to ELK/OpenSearch.
- **Traces.** OpenTelemetry SDK; traces in Jaeger.
- **Errors.** Sentry for frontend and backend, alerting via Slack and PagerDuty.

6.3 CI/CD

- **GitHub Actions** per service: build → unit test → SonarCloud → container build → vulnerability scan → push to ECR → deploy to dev.
- **Argo CD** for GitOps deployments to staging and prod.
- **Conventional commits**, no `var`, named functions, named constants, magic strings forbidden — enforced by lint and code review.

6.4 Integration with Existing Public IT

External system	Integration	Direction
Carpeta Ciudadana Digital (Colombian gov SSO)	OIDC federation	Citizens log in with their government account
Municipal GIS layer	REST consumption + map overlay	Read public lighting / waste routes
Open Data Portal	Public REST API + nightly CSV dump	Outbound transparency
Internal HR system	SCIM / nightly batch	Sync municipal staff into Identity Service
Existing waste-management contractor	Outbound webhooks	Notify contractors of new pickup requests

7. Trade-offs and Architectural Decisions

ADR	Decision	Rationale
ADR-01	Microservices over monolith	Independent scaling per office, parallelisable team work, public-sector procurement often replaces individual modules over time
ADR-02	Kafka over synchronous REST for notifications	Decouples slow gateways (SMS, email) from the user-facing submit flow; resilience
ADR-03	PostgreSQL as primary OLTP	Mature, supports PostGIS for geolocation, lower cost than DynamoDB for relational workloads
ADR-04	Auth0 over self-hosted Keycloak	Faster time-to-market, managed JWKS rotation, social login built-in; revisit at scale
ADR-05	React + Vite + PWA over native mobile	One codebase, offline-capable for low-connectivity neighborhoods, much lower cost

ADR	Decision	Rationale
ADR-06	Mermaid for diagrams (not Draw.io binaries)	Diagrams as code, reviewable in PRs, render natively on GitHub

8. Glossary

- **PQRS** — *Petición, Queja, Reclamo o Sugerencia*. The Colombian legal category that this platform serves.
- **Citizen** — End user, resident of the municipality.
- **Staff** — Municipal employee handling requests.
- **Tracking ID** — Public, opaque code (PQRS-2026-00042) for following a request without an account.
- **SSO** — Single Sign-On; here usually the Colombian *Carpeta Ciudadana Digital*.
- **Story Point (SP)** — Relative effort unit, Fibonacci scale.
- **Velocity** — Story points completed per sprint, averaged over recent sprints.

Deliverable 2 — Agile Product Backlog

Project: CivicLink **Estimation method:** Planning Poker, Fibonacci scale (1, 2, 3, 5, 8, 13) **Total backlog:** 218 SP across 51 user stories in 9 epics **Velocity assumption:** ~45 SP per 2-week sprint (steady-state) **Planned envelope:** 5 sprints (10 weeks)

1. Story-Point Reference Stories

So estimates stay grounded, the team agreed on these references:

SP	Reference Story	Why
1	“Add a static About page”	No backend, no validation
2	“Add a ‘remember me’ checkbox to login”	Trivial frontend + minor backend tweak
3	“Profile edit form (name, phone)”	Form + validation + 1 endpoint
5	“Email-based password reset”	Multi-step flow, email integration

SP	Reference Story	Why
8	“Pin a request location on a map with reverse geocoding”	New library, geo logic, mobile UX
13	“Real-time admin dashboard with live KPIs”	WebSockets + warehouse query + visualisation

Anything >13 SP is split before being committed to a sprint.

2. MoSCoW Prioritization

- **Must** — without this, the platform cannot replace the manual PQRS process.
- **Should** — strongly expected, but the system can launch without it for one cycle.
- **Could** — nice to have, scheduled if velocity allows.
- **Won’t (this release)** — scoped out explicitly to control budget.

3. Epics Overview

ID	Epic	Stories	SP	MoSCoW
E1	Citizen Onboarding & Authentication	6	21	Must
E2	Request Submission	8	34	Must
E3	Request Tracking	6	21	Must
E4	Communication	4	16	Should
E5	Admin Operations	8	39	Must
E6	Reports & Analytics	5	26	Should
E7	Notifications	4	16	Should
E8	Integrations	4	21	Could
E9	Platform & Quality (NFR)	6	24	Must
	Total	51	218	

4. Detailed Backlog

Epic E1 — Citizen Onboarding & Authentication (21 SP)

ID	User Story	SP	Priority
US-01	As a citizen , I want to register an account with email and password so that I can submit and track requests.	5	Must
US-02	As a citizen , I want to log in with email/password so that I can access my dashboard.	3	Must
US-03	As a citizen , I want to log in with Google so that I avoid creating yet another password.	5	Must
US-04	As a citizen , I want to recover my password by email so that I can regain access if I forget it.	3	Must
US-05	As a citizen , I want to edit my profile (name, phone, address) so that the city has accurate contact info.	3	Must
US-06	As a citizen , I want to verify my email so that my account is trustworthy.	2	Must

Acceptance criteria sample (US-01). Given a new visitor, when they submit a valid email and password (≥ 8 chars, 1 number, 1 symbol), then a confirmation email is sent and the account is in **pending_verification** until clicked. Existing emails return 409.

Epic E2 — Request Submission (34 SP)

ID	User Story	SP	Priority
US-07	As a citizen , I want to submit a request with a title and description so that the city knows what to fix .	5	Must
US-08	As a citizen , I want to pick a category (lighting, waste, infrastructure, noise, other) so that the right department gets it .	3	Must
US-09	As a citizen , I want to attach up to 5 photos so that staff see the actual issue .	5	Must
US-10	As a citizen , I want to pin the location on a map so that staff find it quickly .	8	Must

ID	User Story	SP	Priority
US-11	As a citizen , I want to submit anonymously for sensitive issues so that I'm not exposed.	5	Should
US-12	As a citizen , I want to save a draft so that I can finish later.	3	Should
US-13	As a citizen , I want inline form validation so that I know what's missing before I submit.	2	Must
US-14	As a citizen , I want a tracking ID receipt (e.g. PQRS-2026-00042) so that I can follow up without an account.	3	Must

Epic E3 — Request Tracking (21 SP)

ID	User Story	SP	Priority
US-15	As a citizen , I want to see all my requests in a list so that I have one place to look.	3	Must

ID	User Story	SP	Priority
US-16	As a citizen , I want to see the current status of each request (submitted, in review, in progress, resolved, closed) so that I know where it stands.	3	Must
US-17	As a citizen , I want to view the full timeline of a request so that I see every step taken.	5	Must
US-18	As a citizen , I want to see which department and staff member is handling my case so that I know who's responsible.	2	Should
US-19	As a citizen , I want to track a request via a public link with the tracking ID so that I don't always need to log in.	5	Must
US-20	As a citizen , I want to filter and search my requests so that I find the one I care about.	3	Should

Epic E4 — Communication (16 SP)

ID	User Story	SP	Priority
US-21	As a citizen , I want to send messages to staff about my request so that I can clarify or add info.	5	Should
US-22	As staff , I want to reply to citizens through the platform so that conversation history is preserved.	5	Should
US-23	As a citizen , I want to be alerted when staff replies so that I respond promptly.	3	Should
US-24	As a citizen , I want to rate the resolution after closing so that the city improves.	3	Should

Epic E5 — Admin Operations (39 SP)

ID	User Story	SP	Priority
US-25	As an admin , I want a dashboard listing all open requests so that I see the full operational load.	5	Must
US-26	As an admin , I want to filter by department, urgency, location, and status so that I focus on what's relevant.	5	Must
US-27	As a department head , I want to assign requests to specific staff so that work is distributed fairly.	8	Must
US-28	As staff , I want to update the status of my requests so that citizens see progress.	3	Must
US-29	As staff , I want to add internal notes invisible to citizens so that the team coordinates.	3	Must
US-30	As an admin , I want overdue requests automatically escalated so that nothing rots silently.	5	Must

ID	User Story	SP	Priority
US-31	As an admin , I want to manage department configurations (name, area, SLA, auto-assignment rules) so that the org evolves without code changes .	5	Must
US-32	As an admin , I want to manage roles and permissions so that access matches responsibility .	5	Must

Epic E6 — Reports & Analytics (26 SP)

ID	User Story	SP	Priority
US-33	As an admin , I want a KPI dashboard (open requests, avg resolution time, SLA compliance, satisfaction) so that I run the city by data .	8	Should
US-34	As an admin , I want per-department performance reports so that I identify bottlenecks .	5	Should

ID	User Story	SP	Priority
US-35	As an admin , I want request distribution maps and charts by category and zone so that I plan resources .	5	Should
US-36	As an admin , I want to export reports to PDF and Excel so that I share them with the council .	3	Should
US-37	As a citizen / journalist , I want a public transparency dashboard so that the city is held accountable .	5	Should

Epic E7 — Notifications (16 SP)

ID	User Story	SP	Priority
US-38	As a citizen , I want email notifications on every status change so that I'm informed .	5	Should
US-39	As a citizen , I want SMS notifications for urgent updates so that I don't miss them .	5	Should

ID	User Story	SP	Priority
US-40	As a citizen , I want in-app notifications so that I see updates next time I open the portal.	3	Should
US-41	As a citizen , I want to manage notification preferences so that I'm not spammed.	3	Should

Epic E8 — Integrations (21 SP)

ID	User Story	SP	Priority
US-42	As a citizen , I want to log in with Carpeta Ciudadana Digital (Colombian government SSO) so that I reuse my official identity.	8	Could
US-43	As an admin , I want the platform to consume the municipal GIS layer of public lighting so that requests are validated against the network.	5	Could

ID	User Story	SP	Priority
US-44	As a journalist / developer , I want a public REST API so that open data is consumable .	5	Could
US-45	As an admin , I want outbound webhooks to external contractors so that partners are auto-notified .	3	Could

Epic E9 — Platform & Quality / NFR (24 SP)

ID	User Story	SP	Priority
US-46	As DevOps , I want CI/CD pipelines per service so that deployments are automated and auditable .	5	Must
US-47	As DevOps , I want centralised metrics, logs, and traces so that incidents are diagnosable .	5	Must
US-48	As a security officer , I want OWASP Top-10 mitigations and dependency scanning so that the platform stays safe .	5	Must

ID	User Story	SP	Priority
US-49	As a developer , I want feature flags so that risky changes ship safely .	3	Should
US-50	As a citizen , I want a fully mobile-friendly PWA so that I use the platform from any phone .	3	Must
US-51	As an accessibility advocate , I want WCAG 2.1 AA compliance so that every citizen is included .	3	Must

5. Sprint Distribution

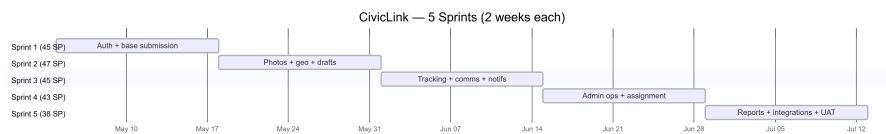


Figure 4: Sprint Distribution Diagram

Sprint	Stories included	SP
1	US-01, US-02, US-04, US-06, US-07, US-08, US-13, US-14, US-46, US-50	30 + buffered NFR
1 (planned)	... reaches 45 SP by adding US-03 (5) and US-25 (5)	45

Sprint	Stories included	SP
2	US-09, US-10, US-11, US-12, US-15, US-16, US-26	47
3	US-17, US-19, US-20, US-21, US-22, US-23, US-38, US-39, US-40	45
4	US-27, US-28, US-29, US-30, US-31, US-32, US-05, US-18	43
5	US-33, US-34, US-35, US-36, US-37, US-24, US-41, US-47, US-48, US-49, US-51	38

The **Could** epic (E8) is deliberately *not* committed to a sprint; it is funded by leftover velocity and an explicit 5 SP buffer in Sprint 5. If velocity beats expectations, US-42 → US-45 are pulled in.

6. Definition of Ready (DoR)

A story enters a sprint only when:

1. INVEST criteria met (Independent, Negotiable, Valuable, Estimable, Small, Testable).
2. Acceptance criteria written in Given/When/Then.
3. UX mockup attached if UI-touching.
4. ≤ 13 SP.
5. Dependencies identified and unblocked.

7. Definition of Done (DoD)

A story is **done** when:

1. Code merged to **main** via PR with at least one approval.
2. Unit + integration tests passing, coverage $\geq 80\%$.
3. Lint and SonarCloud quality gate green.
4. Deployed to **dev** and smoke-tested.
5. Documentation (README, ADR, API docs) updated.
6. Demo'd in sprint review and accepted by Product Owner.

Deliverable 3 — Development Budget

Project: CivicLink **Method:** Velocity-based budgeting (Scrum) **Currency:** USD (with COP equivalents at indicative rate) **Date of estimate:** April 2026

1. Method

Per the workshop reference text, the budget is derived as:

$$\text{Budget} = \text{Cost per Sprint} \times \text{Number of Sprints} + \text{Tools} + \text{Equipment} + \text{Contingency}$$

with effort estimated in **story points** and converted to elapsed sprints using **velocity**:

$$\text{Number of Sprints} = \lceil \text{Total Story Points} / \text{Velocity} \rceil$$

Story points are not hours, but converted to **role-hours per sprint** using each role's allocation in the team. Hourly rates are then applied per role.

2. Inputs

Input	Value	Source
Total backlog	218 SP	Backlog
Steady-state velocity	45 SP / sprint	Industry benchmark for a 6-dev team, validated in retrospectives
Sprint length	2 weeks (10 working days, 80 working hours per FTE)	Standard Scrum
Number of sprints	$\lceil 218 / 45 \rceil = 5$	Computed
Project duration	10 weeks (+1-week Sprint 0 for setup)	Computed
Team FTE composition	7 roles, mixed allocation	Section 3

3. Team and Allocations

Role	Allocation	Hours over 10 weeks	Why this allocation
Project Manager	60%	240	Ceremonies, stakeholder mgmt, budget tracking — not hands-on coding
Tech Lead / Solution Architect	100%	400	Architecture, code reviews, mentoring, hardest stories
Frontend Developer #1	100%	400	Citizen Portal
Frontend Developer #2	100%	400	Admin Dashboard
Backend Developer #1	100%	400	requests-service, departments-service
Backend Developer #2	100%	400	identity-service, notifications-service, reports-service
DevOps Engineer	80%	320	CI/CD, infra, observability, release engineering
QA Engineer	100%	400	Test plans, automation, UAT support
UX/UI Designer	50%	200	Front-loaded in Sprints 0–2, lighter from Sprint 3
Total	—	3,160	

Note. The PM, Tech Lead, DevOps, QA, and UX roles can be staffed by more than one person at lower allocations — the totals (hours and cost) are what matter for the budget.

4. Hourly Rates (Colombian market, 2026)

Rates are blended (mid + senior) and consistent with public salary surveys for Colombia. They are **fully loaded** (gross salary + social contributions + overhead).

Role	Hourly Rate (USD)	Hourly Rate (COP @ 4,200)
Project Manager	\$40	\$168,000
Tech Lead / Solution Architect	\$60	\$252,000
Frontend Developer (blended)	\$35	\$147,000
Backend Developer (blended)	\$38	\$159,600
DevOps Engineer	\$45	\$189,000
QA Engineer	\$28	\$117,600
UX/UI Designer	\$35	\$147,000

5. Labor Cost Breakdown

Role	Hours	Rate (USD)	Cost (USD)
Project Manager (60%)	240	\$40	\$9,600
Tech Lead / Architect	400	\$60	\$24,000
Frontend Dev #1	400	\$35	\$14,000
Frontend Dev #2	400	\$35	\$14,000
Backend Dev #1	400	\$38	\$15,200
Backend Dev #2	400	\$38	\$15,200
DevOps Engineer (80%)	320	\$45	\$14,400
QA Engineer	400	\$28	\$11,200
UX/UI Designer (50%)	200	\$35	\$7,000
Subtotal labor	3,160		\$124,600

6. Other Costs

6.1 Tools and Licenses (10 weeks)

Item	Tier	Cost (USD)
GitHub Team	Per seat × 9	\$200
Auth0 Essentials	Up to 1k MAUs	\$500
Atlassian Jira + Confluence	10 seats	\$300
Figma	3 editor seats	\$150
Sentry Team	1 organization	\$300
Datadog / Grafana Cloud Pro	5 hosts	\$500

Item	Tier	Cost (USD)
AWS dev + staging (compute, RDS, S3, MSK basic, CloudWatch)	mixed	\$2,500
Domains, SSL, miscellaneous	—	\$200
Subtotal		\$4,650

6.2 Equipment and Onboarding

Item	Cost (USD)
Hardware top-up / peripherals (3 × \$500 average)	\$1,500
Onboarding, training material, kickoff workshop	\$500
Subtotal	\$2,000

6.3 Contingency Reserve

15% of (labor + tools + equipment) = $0.15 \times (\$124,600 + \$4,650 + \$2,000) =$
\$19,688.

Contingency covers: scope creep on the **Could** epic (E8), velocity dips during the first sprint, surprises in the Carpeta Ciudadana integration, and minor rate variations.

7. Total Project Cost

Category	Amount (USD)	Amount (COP @ 4,200)
Labor	\$124,600	COP 523,320,000
Tools & licenses	\$4,650	COP 19,530,000
Equipment & onboarding	\$2,000	COP 8,400,000
Contingency (15%)	\$19,688	COP 82,689,600
Total (point estimate)	\$150,938	~ COP 633,940,000

7.1 Range (sensitivity)

Applying $\pm 15\%$ to capture rate volatility, velocity uncertainty, and scope drift:

Scenario	Total (USD)	Total (COP)
Optimistic (P10)	\$128,300	COP 538,860,000
Likely (P50)	\$150,938	COP 633,940,000
Pessimistic (P90)	\$173,580	COP 729,030,000

7.2 Cost per Sprint

Cost per Sprint $\approx \$124,600 / 5 = \$24,920$ (labor only)

Fully loaded per sprint $\approx \$150,938 / 5 = \$30,188$

This means each sprint delivers ≈ 45 SP for $\sim \$30k \rightarrow \sim \mathbf{\$670}$ per story point (fully loaded). Useful for change requests: a new 5 SP story costs roughly \$3,350.

8. Sensitivity and Trade-offs

Lever	Impact
Drop the Could epic (E8, 21 SP)	Saves $\sim \$14,000$ ($\approx$ half a sprint)
Cut UX/UI from 50% to 25%	Saves \$3,500 but risks usability
Replace one Senior dev with a Mid dev ($-\$10/h$)	Saves $\sim \$4,000$, possibly +1 sprint
Add a sixth sprint for hardening	$+\$30k$, raises confidence to $\sim 95\%$
Move from EKS managed control plane to self-hosted K3s	Saves $\sim \$700$ in cloud, adds DevOps load

9. Assumptions

1. **Team is fully staffed from day one** of Sprint 1; Sprint 0 is shared across roles and budgeted within the labor totals.
2. **Velocity stabilises at 45 SP** by Sprint 2; Sprint 1 plans for ~ 45 with low buffer because the team is experienced.
3. **No travel costs**; team is co-located in Bogotá or fully remote with daily overlap.
4. **Production operations are out of scope** of this project budget. Running CivicLink in production is a separate operations budget owned by the municipality.
5. **Auth0 Essentials tier** is sufficient for the build phase. Production volume might require a larger plan.
6. **Government SSO integration** (US-42) requires no paid integration — only OIDC config — and is treated as a Could.
7. **Hourly rates** reflect the Colombian market in 2026; mid+senior blend.

8. **Photo storage at MVP scale** stays under 50 GB — S3 cost is negligible (<\$10/month) and is bundled in the cloud line.

10. Risk Register (cost-relevant)

ID	Risk	Likelihood	Impact (USD)	Mitigation
R1	Carpeta Ciudadana integration delayed or paid	Medium	+\$8,000	Treated as Could; buffered
R2	Velocity dips below 35 SP for 2 sprints	Medium	+\$25,000	Buffer + scope re-prioritisation
R3	A senior dev leaves mid-project	Low	+\$10,000	Tech Lead absorbs; +1 sprint
R4	Cloud cost spike (load tests in staging)	Low	+\$2,000	Rate-limit load tests, schedule off-hours
R5	Auth0 needs Professional plan early	Low	+\$1,500	Cap MAUs; switch tier if needed
R6	Currency volatility (COP/USD)	Medium	±10%	Lock contracts in COP for local team
R7	Scope creep from stakeholders	High	+\$15,000	MoSCoW + change-request priced at \$670/SP

11. Acceptance of the Estimate

This budget is presented as a **fixed-time, scoped backlog** plan: the timeline (10 weeks + 1 setup) and the **Must** scope are committed; **Should** is expected; **Could** is conditional. If the municipality fixes the budget instead of the scope, the team will commit only the **Must** epics first and prioritise **Should** until

budget is exhausted, in line with Scrum’s *fixed-budget* mode described in the workshop reference.

12. Quick Reference

Question	Answer
How long?	10 weeks (5 sprints × 2 weeks) + 1 week Sprint 0
How big a team?	7 roles, 3,160 total hours
How much?	USD \$150,938 point estimate, \$128k – \$174k range
Per sprint?	~\$30,188 fully loaded
Per story point?	~\$670 fully loaded
What’s not included?	Production operations, third-party SaaS production tiers, marketing

User Story Map — CivicLink

User story mapping organises the backlog around the **user journey** (horizontal backbone) and **release slices** (vertical layers). It complements the flat backlog by making *what each release ships to whom* visible at a glance.

The backbone reads like the citizen’s narrative: **Discover → Sign in → Submit → Track → Communicate → Get resolved → Be heard**.

1. Two-axis Map

USER ACTIVITIES (Backbone)							
Discover	Sign in	Submit	Track	Communicate	Resolution	Be heard	
===== RELEASE 1 -- MVP (Sprint 1-3) =====							
Land page	Email/pwd	Title+desc	My list	--	Status	--	
About PQRS	Google	Category	Status		change		
	Verify	Photos	Timeline				
	Reset	Geolocation	Public				
		Tracking ID	link				
		Drafts					
===== RELEASE 2 -- VALUE (Sprint 4) =====							

	Profile edit	Anonymous submission	Filters Search Dept info	Citizen→Staff Staff→Citizen Reply alerts	Assignment Internal notes Escalation Roles Departments	
+===== RELEASE 3 -- INSIGHT (Sprint 5) =====+						
Public trans- parency portal	--	--		Rating on close		KPI dashbd Dept perf Heatmap Export
+===== RELEASE 4 -- INTEGRATIONS (post-launch) =====+						
	Carpeta Ciudadana SSO	--				Open data API Webhook GIS
+=====						

2. Backbone × User Stories

Activity	Release 1 (MVP)	Release 2 (Value)	Release 3 (Insight)	Release 4 (Integrations)
Discover	Landing + About	—	Public transparency portal (US-37)	—
Sign in	US-01, US-02, US-03, US-04, US-06	US-05	—	US-42
Submit	US-07, US-08, US-09, US-10, US-12, US-13, US-14	US-11	—	—
Track	US-15, US-16, US-17, US-19	US-18, US-20	—	—
Communicate	—	US-21, US-22, US-23	US-24	—

Activity	Release 1 (MVP)	Release 2 (Value)	Release 3 (Insight)	Release 4 (Integrations)
Resolution	US-28, US-38	US-27, US-29, US-30, US-31, US-32, US-39, US-40, US-41	—	US-43, US-45
Be heard	—	US-25, US-26	US-33, US-34, US-35, US-36	US-44
Platform / NFR	US-46, US-47, US-48, US-50, US-51	US-49	—	—

3. Reading the Map

Read across (the backbone) to see the citizen’s full journey at any release. Even Release 1 contains every backbone activity except *Communicate* and *Be heard*, which means the MVP delivers an end-to-end “submit → track → resolve” loop — a real release, not a demo.

Read down (each column) to see how an activity grows over releases. *Submit*, for example, gets richer over time without changing the user’s mental model.

Use the map to negotiate. If the budget shrinks, drop the **bottom row** (Release 4) — every Activity above it still tells a complete story. This is the central virtue of mapping over a flat backlog: cuts are visible and humane.